



Figure 1: Keystroke causes interrupt

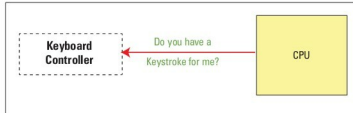


Figure 2: Polling-based interrupt handling



Figure 3: Keystroke passed to the CPU

the next keystroke arrives. The keystroke is passed to the CPU by putting it in the keyboard buffer. So, the keyboard controller keeps on passing the keystroke input to the CPU, but how does the CPU attend to it? The CPU is not at the disposal of the keyboard controller; it is usually busy doing several other operations. So, we need some mechanism to indicate to the CPU that a keystroke has arrived. How is this done? There are two approaches to making sure that the CPU pays attention:

- Polling-based
- Interrupt-based

The polling-based system

We can think of polling as akin to having a phone without the capability to ring aloud. With such a phone, you would have to periodically pick up the receiver to check if there is a call coming through. In a similar fashion, in a polled system, the CPU periodically suspends its tasks and asks the keyboard controller if there is a keystroke (see Figure 2). If there is a keystroke, it forwards it to the CPU (Figure 3). If there is no keystroke (Figure 4), it conveys this to the CPU, and the latter continues with its operations. In either case, the CPU then returns to what it was doing. After some time has passed, it again checks the keyboard controller.

Interrupt-based systems

With an interrupt-based system, the CPU is designed to accept interrupts—input signals from other circuits that ask it to pay attention, as depicted in Figure 5. Examining the same example that we discussed earlier, when the keyboard controller receives a keystroke, it places a signal for the CPU,



Figure 4: No keystroke for CPU

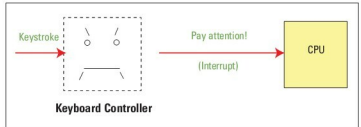


Figure 5: Interrupt-based approach

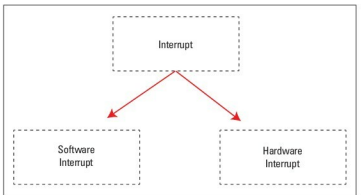


Figure 6: Types of interrupts

asking it to pay attention. The CPU senses the interrupt and puts its current work aside, signalling the keyboard controller that it's ready to accept the keystroke. In contrast to the analogy of a telephone without a ring, the interrupt signal is similar to a telephone's ring. We can continue with our work, without bothering to check every now and then since the bell notifies us of an incoming call.

As you would agree, polling requires less support hardware, but the CPU spends a good amount of time in checking if any of the polled peripherals need attention. Thus, an interrupt-based system is usually preferred over a polling-based system.

Broadly, interrupts are classified into hardware interrupts and software interrupts.

Software interrupts

A software interrupt is one that's generated within a processor by executing an instruction. Examples of software interrupts are system calls (like 'fork' and 'kill' in UNIX).

Hardware interrupts

A hardware interrupt is a signal that is created and sent to the CPU because of some action in/on a hardware device. For example, keystrokes and mouse movements cause hardware interrupts. A hardware interrupt causes the processor to